

EVM链 Golang API调用文档

本文档整理了在生产环境中常用的EVM系区块链（以太坊、BSC、Polygon等）Golang API调用方式，包含详细的入参、出参说明和数据结构化存储方案。

目录

- [环境准备与初始化](#)
- [账户与余额操作](#)
- [交易操作](#)
- [智能合约交互](#)
- [事件监听](#)
- [区块数据获取](#)
- [数据结构化存储](#)
- [生产环境最佳实践](#)

1. 环境准备与初始化

1.1 安装依赖

```
go get github.com/ethereum/go-ethereum
go get github.com/ethereum/go-ethereum/ethclient
go get github.com/ethereum/go-ethereum/common
go get github.com/ethereum/go-ethereum/core/types
```

1.2 客户端初始化

功能说明：创建与EVM链的RPC连接，支持多节点负载均衡和故障切换。

核心流程：

- 配置解析**：读取链ID、RPC节点列表、超时时间等配置参数
- 节点连接**：按顺序尝试连接配置的RPC节点，找到第一个可用节点
- 连接管理**：建立持久连接并支持动态切换到备用节点
- 错误处理**：当前节点不可用时自动切换到下一个节点

技术要点：

- 使用 `context.WithTimeout` 控制连接超时
- 实现轮询机制选择可用节点
- 支持WebSocket和HTTP两种连接方式
- 连接失败时的优雅降级处理

```
package main

import (
    "context"
```

```

    "fmt"
    "log"
    "time"

    "github.com/ethereum/go-ethereum/ethclient"
)

// ChainConfig 链配置
type ChainConfig struct {
    ChainID      int64      `json:"chain_id"`      // 链ID (以太坊:1, BSC:56, Polygon:137)
    RPCUrls      []string   `json:"rpc_urls"`      // RPC节点地址列表
    WSUrls       []string   `json:"ws_urls"`       // WebSocket节点地址列表
    Timeout      int        `json:"timeout"`       // 超时时间 (秒)
    RetryCount   int        `json:"retry_count"`   // 重试次数
}

// EVMClient EVM客户端包装
type EVMClient struct {
    client      *ethclient.Client
    config      *ChainConfig
    ctx         context.Context
    cancel      context.CancelFunc
    currentRPC  int // 当前使用的RPC索引
}

// NewEVMClient 创建EVM客户端
// 入参:
//   - config: 链配置信息
// 出参:
//   - *EVMClient: 客户端实例
//   - error: 错误信息
func NewEVMClient(config *ChainConfig) (*EVMClient, error) {
    ctx, cancel := context.WithTimeout(context.Background(),
time.Duration(config.Timeout)*time.Second)

    // 尝试连接第一个可用节点
    var client *ethclient.Client
    var err error

    for i, rpcURL := range config.RPCUrls {
        client, err = ethclient.DialContext(ctx, rpcURL)
        if err == nil {
            log.Printf("成功连接到节点: %s", rpcURL)
            return &EVMClient{
                client:      client,
                config:      config,
                ctx:         ctx,
                cancel:      cancel,
                currentRPC: i,
            }, nil
        }
    }
    log.Printf("连接节点失败 %s: %v", rpcURL, err)
}

```

```

    }

    cancel()
    return nil, fmt.Errorf("所有RPC节点连接失败")
}

// Close 关闭客户端连接
func (c *EVMClient) Close() {
    if c.client != nil {
        c.client.Close()
    }
    if c.cancel != nil {
        c.cancel()
    }
}

// SwitchRPC 切换到下一个RPC节点
func (c *EVMClient) SwitchRPC() error {
    c.currentRPC = (c.currentRPC + 1) % len(c.config.RPCUrls)
    nextRPC := c.config.RPCUrls[c.currentRPC]

    client, err := ethclient.DialContext(c.ctx, nextRPC)
    if err != nil {
        return fmt.Errorf("切换RPC失败 %s: %w", nextRPC, err)
    }

    c.client.Close()
    c.client = client
    log.Printf("成功切换到节点: %s", nextRPC)
    return nil
}

```

使用示例:

```

func main() {
    config := &ChainConfig{
        ChainID: 56, // BSC主网
        RPCUrls: []string{
            "https://bsc-dataseed1.binance.org",
            "https://bsc-dataseed2.binance.org",
            "https://bsc-dataseed3.binance.org",
        },
        Timeout: 30,
        RetryCount: 3,
    }

    client, err := NewEVMClient(config)
    if err != nil {
        log.Fatal(err)
    }
    defer client.Close()
}

```

```
// 使用客户端...  
}
```

2. 账户与余额操作

2.1 获取账户余额

功能说明： 查询指定地址的原生代币余额（ETH/BNB/MATIC等）。

核心流程：

1. 地址验证：将十六进制地址字符串转换为 `common.Address` 类型
2. 区块高度获取：获取当前最新区块高度作为查询基准
3. 余额查询：调用 `BalanceAt` 方法查询指定区块高度的账户余额
4. 单位转换：将wei单位的余额转换为ether单位便于显示

技术要点：

- 余额以wei为最小单位（1 ether = 10^{18} wei）
- 使用 `big.Int` 处理大数值避免精度丢失
- 指定区块高度确保数据一致性
- 支持历史余额查询

```
import (  
    "context"  
    "math/big"  
  
    "github.com/ethereum/go-ethereum/common"  
    "github.com/ethereum/go-ethereum/ethclient"  
)  
  
// BalanceInfo 余额信息  
type BalanceInfo struct {  
    Address      string    `json:"address"`      // 地址  
    Balance      *big.Int  `json:"balance"`      // 余额 (wei)  
    BalanceEther string    `json:"balance_ether"` // 余额 (ether)  
    BlockNumber  uint64    `json:"block_number"` // 查询时的区块高度  
}  
  
// GetBalance 获取账户余额  
// 入参：  
//   - client: ethclient客户端  
//   - address: 账户地址（十六进制字符串）  
// 出参：  
//   - *BalanceInfo: 余额信息  
//   - error: 错误信息  
func GetBalance(client *ethclient.Client, address string) (*BalanceInfo, error) {  
    ctx := context.Background()
```

```

// 获取当前区块高度
blockNumber, err := client.BlockNumber(ctx)
if err != nil {
    return nil, fmt.Errorf("获取区块高度失败: %w", err)
}

// 查询余额
account := common.HexToAddress(address)
balance, err := client.BalanceAt(ctx, account, big.NewInt(int64(blockNumber)))
if err != nil {
    return nil, fmt.Errorf("查询余额失败: %w", err)
}

// 转换为ether单位 (1 ether = 10^18 wei)
fbalance := new(big.Float).SetInt(balance)
ethValue := new(big.Float).Quo(fbalance, big.NewFloat(1e18))

return &BalanceInfo{
    Address:      address,
    Balance:      balance,
    BalanceEther: ethValue.String(),
    BlockNumber:  blockNumber,
}, nil
}

```

2.2 获取ERC20代币余额

功能说明：查询指定地址的ERC20代币余额。

核心流程：

1. **ABI解析**：解析ERC20标准合约的ABI定义，获取方法签名
2. **合约绑定**：创建合约实例，绑定到指定的代币合约地址
3. **批量查询**：并行调用 `balanceOf`、`decimals`、`symbol`、`name` 等方法
4. **数据整合**：将查询结果封装为结构化的代币余额信息

技术要点：

- 使用 `bind.NewBoundContract` 创建合约绑定
- 通过 `CallOpts` 设置只读调用参数
- ERC20标准方法的统一调用接口
- 错误处理确保部分查询失败不影响整体结果

```

import (
    "github.com/ethereum/go-ethereum/accounts/abi"
    "github.com/ethereum/go-ethereum/accounts/abi/bind"
)

// ERC20Balance ERC20代币余额
type ERC20Balance struct {
    TokenAddress string `json:"token_address"` // 代币合约地址
    HolderAddress string `json:"holder_address"` // 持有者地址
}

```

```

    Balance      *big.Int `json:"balance"`      // 余额
    Decimals      uint8   `json:"decimals"`      // 精度
    Symbol        string  `json:"symbol"`        // 代币符号
    Name          string  `json:"name"`          // 代币名称
}

// GetERC20Balance 获取ERC20代币余额
// 入参:
//   - client: ethclient客户端
//   - tokenAddress: 代币合约地址
//   - holderAddress: 持有者地址
// 出参:
//   - *ERC20Balance: 代币余额信息
//   - error: 错误信息
func GetERC20Balance(client *ethclient.Client, tokenAddress, holderAddress string)
(*ERC20Balance, error) {
    ctx := context.Background()

    // ERC20 ABI定义
    erc20ABI := `[
        {"constant":true,"inputs":
[{"name":"_owner","type":"address"}],"name":"balanceOf","outputs":
[{"name":"balance","type":"uint256"}],"type":"function"},
        {"constant":true,"inputs":[],"name":"decimals","outputs":
[{"name":"","type":"uint8"}],"type":"function"},
        {"constant":true,"inputs":[],"name":"symbol","outputs":
[{"name":"","type":"string"}],"type":"function"},
        {"constant":true,"inputs":[],"name":"name","outputs":
[{"name":"","type":"string"}],"type":"function"}
    ]`

    // 解析ABI
    parsedABI, err := abi.JSON(strings.NewReader(erc20ABI))
    if err != nil {
        return nil, fmt.Errorf("解析ABI失败: %w", err)
    }

    token := common.HexToAddress(tokenAddress)
    holder := common.HexToAddress(holderAddress)

    // 创建合约绑定
    boundContract := bind.NewBoundContract(token, parsedABI, client, client, client)

    result := &ERC20Balance{
        TokenAddress: tokenAddress,
        HolderAddress: holderAddress,
    }

    // 查询余额
    var balance *big.Int
    err = boundContract.Call(&bind.CallOpts{Context: ctx}, &balance, "balanceOf", holder)
    if err != nil {

```

```

        return nil, fmt.Errorf("查询余额失败: %w", err)
    }
    result.Balance = balance

    // 查询精度
    var decimals uint8
    err = boundContract.Call(&bind.CallOpts{Context: ctx}, &decimals, "decimals")
    if err != nil {
        return nil, fmt.Errorf("查询精度失败: %w", err)
    }
    result.Decimals = decimals

    // 查询符号
    var symbol string
    err = boundContract.Call(&bind.CallOpts{Context: ctx}, &symbol, "symbol")
    if err != nil {
        return nil, fmt.Errorf("查询符号失败: %w", err)
    }
    result.Symbol = symbol

    // 查询名称
    var name string
    err = boundContract.Call(&bind.CallOpts{Context: ctx}, &name, "name")
    if err != nil {
        return nil, fmt.Errorf("查询名称失败: %w", err)
    }
    result.Name = name

    return result, nil
}

```

3. 交易操作

3.1 发送原生代币转账

功能说明：发送ETH/BNB等原生代币到指定地址。

核心流程：

1. **私钥解析**：将十六进制私钥字符串转换为ECDSA私钥对象
2. **地址推导**：从私钥推导出发送者的公钥地址
3. **Nonce获取**：查询发送者当前的pending nonce值防止交易冲突
4. **Gas估算**：自动估算当前网络的合理Gas价格
5. **交易构建**：创建包含转账信息的原始交易对象
6. **交易签名**：使用EIP155标准对交易进行数字签名
7. **广播交易**：将签名后的交易发送到区块链网络

技术要点：

- 使用 `PendingNonceAt` 获取最新nonce避免交易卡住
- 支持手动指定Gas价格或自动估算

- EIP155签名防止重放攻击
- 交易发送后立即返回哈希，无需等待确认

```
import (
    "crypto/ecdsa"

    "github.com/ethereum/go-ethereum/crypto"
)

// TransferRequest 转账请求
type TransferRequest struct {
    PrivateKey string `json:"private_key"` // 发送者私钥（十六进制）
    ToAddress  string `json:"to_address"`   // 接收地址
    Amount     *big.Int `json:"amount"`       // 转账金额（wei）
    GasLimit   uint64   `json:"gas_limit"`    // Gas限制
    GasPrice   *big.Int `json:"gas_price"`    // Gas价格（可选，nil则自动估算）
}

// TransferResponse 转账响应
type TransferResponse struct {
    TxHash      string `json:"tx_hash"`      // 交易哈希
    From        string `json:"from"`         // 发送者地址
    To          string `json:"to"`           // 接收者地址
    Amount      *big.Int `json:"amount"`       // 转账金额
    GasPrice    *big.Int `json:"gas_price"`    // 实际Gas价格
    GasLimit    uint64   `json:"gas_limit"`    // Gas限制
    Nonce       uint64   `json:"nonce"`        // Nonce值
    BlockNumber uint64   `json:"block_number"` // 打包区块号（0表示未确认）
}

// SendTransaction 发送原生代币转账
// 入参：
//   - client: ethclient客户端
//   - chainID: 链ID
//   - req: 转账请求
// 出参：
//   - *TransferResponse: 转账响应
//   - error: 错误信息
func SendTransaction(client *ethclient.Client, chainID *big.Int, req *TransferRequest)
(*TransferResponse, error) {
    ctx := context.Background()

    // 解析私钥
    privateKey, err := crypto.HexToECDSA(req.PrivateKey)
    if err != nil {
        return nil, fmt.Errorf("解析私钥失败: %w", err)
    }

    // 获取发送者地址
    publicKey := privateKey.Public()
    publicKeyECDSA, ok := publicKey.(*ecdsa.PublicKey)
    if !ok {
```

```

        return nil, fmt.Errorf("获取公钥失败")
    }
    fromAddress := crypto.PubkeyToAddress(*publicKeyECDSA)

    // 获取nonce
    nonce, err := client.PendingNonceAt(ctx, fromAddress)
    if err != nil {
        return nil, fmt.Errorf("获取nonce失败: %w", err)
    }

    // 获取或估算Gas价格
    gasPrice := req.GasPrice
    if gasPrice == nil {
        gasPrice, err = client.SuggestGasPrice(ctx)
        if err != nil {
            return nil, fmt.Errorf("估算Gas价格失败: %w", err)
        }
    }
}

// 构建交易
toAddress := common.HexToAddress(req.ToAddress)
tx := types.NewTransaction(
    nonce,
    toAddress,
    req.Amount,
    req.GasLimit,
    gasPrice,
    nil, // data
)

// 签名交易
signedTx, err := types.SignTx(tx, types.NewEIP155Signer(chainID), privateKey)
if err != nil {
    return nil, fmt.Errorf("签名交易失败: %w", err)
}

// 发送交易
err = client.SendTransaction(ctx, signedTx)
if err != nil {
    return nil, fmt.Errorf("发送交易失败: %w", err)
}

return &TransferResponse{
    TxHash:    signedTx.Hash().Hex(),
    From:      fromAddress.Hex(),
    To:        req.ToAddress,
    Amount:    req.Amount,
    GasPrice:  gasPrice,
    GasLimit:  req.GasLimit,
    Nonce:     nonce,
}, nil
}

```

3.2 查询交易状态

功能说明： 查询交易是否被打包及执行状态。

核心流程：

1. 交易收据获取：通过交易哈希获取链上执行收据
2. 交易详情查询：获取交易的完整信息包括发送者、接收者、金额等
3. 确认数计算：对比当前区块高度与交易所在区块计算确认数
4. 状态解析：解析交易执行状态（成功/失败）和Gas消耗情况

技术要点：

- 交易收据包含执行结果和Gas使用情况
- `isPending` 标志判断交易是否还在内存池中
- 确认数反映交易的安全程度（通常6个确认认为安全）
- Status字段：1表示成功，0表示失败

```
// TransactionStatus 交易状态
type TransactionStatus struct {
    TxHash      string    `json:"tx_hash"`           // 交易哈希
    Status      uint64    `json:"status"`           // 状态 (1:成功, 0:失败)
    BlockNumber uint64    `json:"block_number"`     // 区块号
    BlockHash   string    `json:"block_hash"`       // 区块哈希
    GasUsed     uint64    `json:"gas_used"`         // 实际消耗Gas
    From        string    `json:"from"`             // 发送者
    To          string    `json:"to"`               // 接收者
    Value       *big.Int  `json:"value"`            // 转账金额
    Confirmations uint64    `json:"confirmations"`    // 确认数
}

// GetTransactionStatus 查询交易状态
// 入参:
//   - client: ethclient客户端
//   - txHash: 交易哈希
// 出参:
//   - *TransactionStatus: 交易状态
//   - error: 错误信息
func GetTransactionStatus(client *ethclient.Client, txHash string) (*TransactionStatus, error) {
    ctx := context.Background()

    hash := common.HexToHash(txHash)

    // 获取交易收据
    receipt, err := client.TransactionReceipt(ctx, hash)
    if err != nil {
        return nil, fmt.Errorf("获取交易收据失败: %w", err)
    }

    // 获取交易详情
    tx, isPending, err := client.TransactionByHash(ctx, hash)
```

```
if err != nil {
    return nil, fmt.Errorf("获取交易详情失败: %w", err)
}

if isPending {
    return nil, fmt.Errorf("交易待确认中")
}

// 获取当前区块高度计算确认数
currentBlock, err := client.BlockNumber(ctx)
if err != nil {
    return nil, fmt.Errorf("获取当前区块失败: %w", err)
}

confirmations := currentBlock - receipt.BlockNumber.Uint64()

var toAddress string
if tx.To() != nil {
    toAddress = tx.To().Hex()
}

return &TransactionStatus{
    TxHash:      txHash,
    Status:      receipt.Status,
    BlockNumber: receipt.BlockNumber.Uint64(),
    BlockHash:   receipt.BlockHash.Hex(),
    GasUsed:     receipt.GasUsed,
    From:        receipt.From.Hex(),
    To:          toAddress,
    Value:       tx.Value(),
    Confirmations: confirmations,
}, nil
}
```

4. 智能合约交互

4.1 调用合约只读方法

功能说明：调用合约的view/pure方法，不消耗Gas。

核心流程：

1. **ABI解析**：解析合约ABI获取方法定义和参数类型
2. **合约绑定**：创建合约实例并绑定到指定地址
3. **参数准备**：根据ABI定义准备方法调用参数
4. **本地调用**：通过 `Call` 方法在本地节点执行只读操作
5. **结果解析**：解析返回值并转换为Go语言类型

技术要点：

- 只读方法不会改变区块链状态，无需消耗Gas

- 使用 `CallOpts` 可以指定调用的区块高度
- 支持多返回值的方法调用
- 调用失败时返回具体的错误信息

```
// ContractCallRequest 合约调用请求
type ContractCallRequest struct {
    ContractAddress string    `json:"contract_address"` // 合约地址
    ABI              string    `json:"abi"`              // 合约ABI (JSON字符串)
    MethodName       string    `json:"method_name"`      // 方法名
    Params           []interface{} `json:"params"`          // 参数列表
}

// CallContract 调用合约只读方法
// 入参:
//   - client: ethclient客户端
//   - req: 调用请求
// 出参:
//   - []interface{}: 返回值列表
//   - error: 错误信息
func CallContract(client *ethclient.Client, req *ContractCallRequest) ([]interface{}, error) {
    ctx := context.Background()

    // 解析ABI
    parsedABI, err := abi.JSON(strings.NewReader(req.ABI))
    if err != nil {
        return nil, fmt.Errorf("解析ABI失败: %w", err)
    }

    // 创建合约绑定
    contractAddr := common.HexToAddress(req.ContractAddress)
    boundContract := bind.NewBoundContract(contractAddr, parsedABI, client, client, client)

    // 准备输出
    method := parsedABI.Methods[req.MethodName]
    outputs := make([]interface{}, len(method.Outputs))
    outputPtrs := make([]interface{}, len(method.Outputs))

    for i := range outputs {
        outputPtrs[i] = &outputs[i]
    }

    // 调用合约
    err = boundContract.Call(&bind.CallOpts{Context: ctx}, &outputs, req.MethodName, req.Params...)
    if err != nil {
        return nil, fmt.Errorf("调用合约失败: %w", err)
    }

    return outputs, nil
}
```

```
}
```

4.2 发送合约交易

功能说明：调用合约的state-changing方法，需要消耗Gas。

核心流程：

1. 私钥验证：解析并验证发送者私钥的有效性
2. **ABI编码**：根据合约ABI将方法名和参数编码为调用数据
3. 交易构建：创建包含合约调用数据的交易对象
4. **Gas估算**：估算合约方法执行所需的Gas限制
5. 交易签名：使用私钥对交易进行数字签名
6. 交易广播：将签名交易发送到区块链网络

技术要点：

- 使用 `abi.Pack` 方法编码合约调用数据
- 支持向合约发送ETH（payable方法）
- Gas限制设置过低会导致交易失败
- 合约调用失败会消耗Gas但不会改变状态

```
// ContractTransactionRequest 合约交易请求
type ContractTransactionRequest struct {
    PrivateKey      string    `json:"private_key"` // 发送者私钥
    ContractAddress string    `json:"contract_address"` // 合约地址
    ABI             string    `json:"abi"`           // 合约ABI
    MethodName      string    `json:"method_name"`   // 方法名
    Params          []interface{} `json:"params"`        // 参数列表
    Value           *big.Int   `json:"value"`          // 附带的ETH（可选）
    GasLimit        uint64     `json:"gas_limit"`      // Gas限制
    GasPrice        *big.Int   `json:"gas_price"`      // Gas价格（可选）
}

// SendContractTransaction 发送合约交易
// 入参：
//   - client: ethclient客户端
//   - chainID: 链ID
//   - req: 交易请求
// 出参：
//   - *TransferResponse: 交易响应
//   - error: 错误信息
func SendContractTransaction(client *ethclient.Client, chainID *big.Int, req
*ContractTransactionRequest) (*TransferResponse, error) {
    ctx := context.Background()

    // 解析私钥
    privateKey, err := crypto.HexToECDSA(req.PrivateKey)
    if err != nil {
        return nil, fmt.Errorf("解析私钥失败: %w", err)
    }
}
```

```
publicKey := privateKey.Public()
publicKeyECDSA, _ := publicKey.(*ecdsa.PublicKey)
fromAddress := crypto.PubkeyToAddress(*publicKeyECDSA)

// 获取nonce
nonce, err := client.PendingNonceAt(ctx, fromAddress)
if err != nil {
    return nil, fmt.Errorf("获取nonce失败: %w", err)
}

// 获取Gas价格
gasPrice := req.GasPrice
if gasPrice == nil {
    gasPrice, err = client.SuggestGasPrice(ctx)
    if err != nil {
        return nil, fmt.Errorf("估算Gas价格失败: %w", err)
    }
}

// 解析ABI并编码调用数据
parsedABI, err := abi.JSON(strings.NewReader(req.ABI))
if err != nil {
    return nil, fmt.Errorf("解析ABI失败: %w", err)
}

data, err := parsedABI.Pack(req.MethodName, req.Params...)
if err != nil {
    return nil, fmt.Errorf("编码调用数据失败: %w", err)
}

// 构建交易
contractAddr := common.HexToAddress(req.ContractAddress)
value := req.Value
if value == nil {
    value = big.NewInt(0)
}

tx := types.NewTransaction(
    nonce,
    contractAddr,
    value,
    req.GasLimit,
    gasPrice,
    data,
)

// 签名并发送
signedTx, err := types.SignTx(tx, types.NewEIP155Signer(chainID), privateKey)
if err != nil {
    return nil, fmt.Errorf("签名交易失败: %w", err)
}
```

```

err = client.SendTransaction(ctx, signedTx)
if err != nil {
    return nil, fmt.Errorf("发送交易失败: %w", err)
}

return &TransferResponse{
    TxHash:    signedTx.Hash().Hex(),
    From:      fromAddress.Hex(),
    To:        req.ContractAddress,
    Amount:    value,
    GasPrice:  gasPrice,
    GasLimit:  req.GasLimit,
    Nonce:     nonce,
}, nil
}

```

5. 事件监听

5.1 监听合约事件（实时）

功能说明：通过WebSocket实时监听合约事件。

核心流程：

1. **WebSocket连接**：建立与区块链节点的WebSocket长连接
2. **过滤器构建**：根据合约地址和事件签名创建日志过滤器
3. **订阅创建**：调用 `SubscribeFilterLogs` 创建事件订阅
4. **事件处理**：在独立协程中处理接收到的事件日志
5. **错误恢复**：监听订阅错误并实现自动重连机制

技术要点：

- 需要WebSocket连接才能实现实时监听
- 使用Topics过滤特定事件类型
- 事件可能因为分叉被标记为 `Removed`
- 订阅会在网络中断时自动断开，需要重连处理

```

import (
    "github.com/ethereum/go-ethereum"
)

// EventSubscription 事件订阅
type EventSubscription struct {
    ContractAddress string `json:"contract_address"` // 合约地址
    EventSignature   string `json:"event_signature"`  // 事件签名
    Topics           []string `json:"topics"`           // 过滤topics
}

// ContractEvent 合约事件
type ContractEvent struct {

```

```

ContractAddress string           `json:"contract_address"` // 合约地址
EventName       string           `json:"event_name"`       // 事件名称
BlockNumber     uint64           `json:"block_number"`     // 区块号
TxHash          string           `json:"tx_hash"`          // 交易哈希
LogIndex        uint             `json:"log_index"`        // 日志索引
Data            map[string]interface{} `json:"data"`            // 事件数据
Removed         bool             `json:"removed"`          // 是否被移除 (分叉)
}

// SubscribeContractEvents 订阅合约事件
// 入参:
//   - client: ethclient客户端 (需要WebSocket连接)
//   - sub: 订阅配置
//   - eventChan: 事件通道
// 出参:
//   - ethereum.Subscription: 订阅实例
//   - error: 错误信息
func SubscribeContractEvents(
    client *ethclient.Client,
    sub *EventSubscription,
    eventChan chan<- *ContractEvent,
) (ethereum.Subscription, error) {

    // 构建过滤查询
    query := ethereum.FilterQuery{
        Addresses: []common.Address{common.HexToAddress(sub.ContractAddress)},
    }

    // 添加topic过滤
    if len(sub.Topics) > 0 {
        topics := make([]common.Hash, len(sub.Topics))
        for i, topic := range sub.Topics {
            topics[i] = common.HexToHash(topic)
        }
        query.Topics = [][]common.Hash{topics}
    }

    // 创建日志通道
    logs := make(chan types.Log)

    // 订阅
    subscription, err := client.SubscribeFilterLogs(context.Background(), query, logs)
    if err != nil {
        return nil, fmt.Errorf("订阅事件失败: %w", err)
    }

    // 处理事件
    go func() {
        for {
            select {
            case err := <-subscription.Err():
                log.Printf("订阅错误: %v", err)
            }
        }
    }()
}

```

```

        return
    case vLog := <-logs:
        event := &ContractEvent{
            ContractAddress: vLog.Address.Hex(),
            BlockNumber:     vLog.BlockNumber,
            TxHash:          vLog.TxHash.Hex(),
            LogIndex:        vLog.Index,
            Removed:         vLog.Removed,
            Data:             make(map[string]interface{}),
        }

        // 这里需要根据ABI解析事件数据
        // 实际使用时需要传入ABI进行解析

        eventChan <- event
    }
}

return subscription, nil
}

```

5.2 查询历史事件

功能说明： 查询指定区块范围内的历史事件。

核心流程：

1. **区块范围设定：** 指定查询的起始和结束区块号
2. **过滤器配置：** 设置合约地址、事件签名等过滤条件
3. **批量查询：** 调用 `FilterLogs` 获取指定范围内的所有匹配事件
4. **数据转换：** 将原始日志数据转换为结构化的事件对象
5. **结果排序：** 按区块号和日志索引排序确保事件顺序

技术要点：

- 查询范围过大可能导致超时，建议分批查询
- 使用Topics数组实现复杂的事件过滤
- 历史事件不会被分叉影响，数据更稳定
- 支持从创世区块开始的完整历史查询

```

// HistoricalEventsRequest 历史事件查询请求
type HistoricalEventsRequest struct {
    ContractAddress string `json:"contract_address"` // 合约地址
    EventSignature   string `json:"event_signature"`  // 事件签名
    FromBlock        uint64 `json:"from_block"`       // 起始区块
    ToBlock          uint64 `json:"to_block"`         // 结束区块
    Topics           []string `json:"topics"`           // 过滤topics
}

// QueryHistoricalEvents 查询历史事件
// 入参：

```

```

// - client: ethclient客户端
// - req: 查询请求
// 出参:
// - []*ContractEvent: 事件列表
// - error: 错误信息
func QueryHistoricalEvents(client *ethclient.Client, req *HistoricalEventsRequest)
([]*ContractEvent, error) {
    ctx := context.Background()

    // 构建查询
    query := ethereum.FilterQuery{
        FromBlock: big.NewInt(int64(req.FromBlock)),
        ToBlock:    big.NewInt(int64(req.ToBlock)),
        Addresses: []common.Address{common.HexToAddress(req.ContractAddress)},
    }

    // 添加topic过滤
    if len(req.Topics) > 0 {
        topics := make([]common.Hash, len(req.Topics))
        for i, topic := range req.Topics {
            topics[i] = common.HexToHash(topic)
        }
        query.Topics = [][]common.Hash{topics}
    }

    // 查询日志
    logs, err := client.FilterLogs(ctx, query)
    if err != nil {
        return nil, fmt.Errorf("查询日志失败: %w", err)
    }

    // 转换为事件
    events := make([]*ContractEvent, 0, len(logs))
    for _, vLog := range logs {
        event := &ContractEvent{
            ContractAddress: vLog.Address.Hex(),
            BlockNumber:       vLog.BlockNumber,
            TxHash:            vLog.TxHash.Hex(),
            LogIndex:          vLog.Index,
            Removed:           vLog.Removed,
            Data:              make(map[string]interface{}),
        }
        events = append(events, event)
    }

    return events, nil
}

```

6. 区块数据获取

6.1 获取最新区块

功能说明： 获取链上最新的区块信息。

核心流程：

1. 最新高度查询：调用 `BlockNumber` 获取当前链的最新区块高度
2. 区块详情获取：根据区块号获取完整的区块信息
3. 交易提取：从区块中提取所有交易的哈希列表
4. 数据封装：将区块信息封装为结构化对象便于使用

技术要点：

- 最新区块可能因为分叉而改变
- 区块包含头部信息和交易列表两部分
- Timestamp是Unix时间戳格式
- GasUsed反映区块的拥堵程度

```
// BlockInfo 区块信息
type BlockInfo struct {
    Number      uint64    `json:"number"`      // 区块号
    Hash        string     `json:"hash"`        // 区块哈希
    ParentHash   string     `json:"parent_hash"`  // 父区块哈希
    Timestamp    uint64    `json:"timestamp"`   // 时间戳
    Miner       string     `json:"miner"`       // 矿工地址
    Difficulty   *big.Int  `json:"difficulty"`   // 难度
    GasLimit     uint64    `json:"gas_limit"`   // Gas限制
    GasUsed      uint64    `json:"gas_used"`    // Gas使用量
    TxCount      int       `json:"tx_count"`    // 交易数量
    Transactions []string  `json:"transactions"` // 交易哈希列表
}

// GetLatestBlock 获取最新区块
// 入参：
//   - client: ethclient客户端
// 出参：
//   - *BlockInfo: 区块信息
//   - error: 错误信息
func GetLatestBlock(client *ethclient.Client) (*BlockInfo, error) {
    ctx := context.Background()

    // 获取最新区块号
    blockNumber, err := client.BlockNumber(ctx)
    if err != nil {
        return nil, fmt.Errorf("获取区块号失败: %w", err)
    }

    // 获取区块详情
    block, err := client.BlockByNumber(ctx, big.NewInt(int64(blockNumber)))
    if err != nil {
        return nil, fmt.Errorf("获取区块失败: %w", err)
    }
}
```

```

// 提取交易哈希
txHashes := make([]string, 0, len(block.Transactions()))
for _, tx := range block.Transactions() {
    txHashes = append(txHashes, tx.Hash().Hex())
}

return &BlockInfo{
    Number:      block.NumberU64(),
    Hash:        block.Hash().Hex(),
    ParentHash:  block.ParentHash().Hex(),
    Timestamp:   block.Time(),
    Miner:       block.Coinbase().Hex(),
    Difficulty:  block.Difficulty(),
    GasLimit:    block.GasLimit(),
    GasUsed:     block.GasUsed(),
    TxCount:     len(block.Transactions()),
    Transactions: txHashes,
}, nil
}

```

6.2 获取指定区块的所有交易

功能说明： 获取某个区块内的所有交易详情。

核心流程：

1. **区块获取：** 根据区块号获取完整区块数据
2. **交易遍历：** 遍历区块中的所有交易对象
3. **发送者解析：** 通过签名恢复每笔交易的发送者地址
4. **数据提取：** 提取交易的关键信息如金额、Gas、数据等
5. **批量封装：** 将所有交易信息封装为统一格式

技术要点：

- 使用 `AsMessage` 方法恢复交易发送者地址
- 合约创建交易的To字段为nil
- Data字段包含合约调用的编码数据
- 交易在区块中的顺序反映了执行顺序

```

// TransactionDetail 交易详情
type TransactionDetail struct {
    Hash          string    `json:"hash"`           // 交易哈希
    From          string    `json:"from"`           // 发送者
    To            string    `json:"to"`             // 接收者
    Value         *big.Int `json:"value"`           // 转账金额
    Gas           uint64   `json:"gas"`             // Gas限制
    GasPrice      *big.Int `json:"gas_price"`       // Gas价格
    Nonce         uint64   `json:"nonce"`           // Nonce
    Data          string    `json:"data"`            // 交易数据（十六进制）
    BlockNumber   uint64   `json:"block_number"`    // 区块号
    BlockHash     string    `json:"block_hash"`      // 区块哈希
}

```

```

}

// GetBlockTransactions 获取区块的所有交易
// 入参:
//   - client: ethclient客户端
//   - blockNumber: 区块号
// 出参:
//   - []*TransactionDetail: 交易详情列表
//   - error: 错误信息
func GetBlockTransactions(client *ethclient.Client, blockNumber uint64)
([]*TransactionDetail, error) {
    ctx := context.Background()

    // 获取区块
    block, err := client.BlockByNumber(ctx, big.NewInt(int64(blockNumber)))
    if err != nil {
        return nil, fmt.Errorf("获取区块失败: %w", err)
    }

    // 解析交易
    transactions := make([]*TransactionDetail, 0, len(block.Transactions()))

    for _, tx := range block.Transactions() {
        // 获取发送者地址
        msg, err := tx.AsMessage(types.NewEIP155Signer(tx.ChainId()), nil)
        if err != nil {
            log.Printf("解析交易消息失败 %s: %v", tx.Hash().Hex(), err)
            continue
        }

        var toAddress string
        if tx.To() != nil {
            toAddress = tx.To().Hex()
        }

        detail := &TransactionDetail{
            Hash:      tx.Hash().Hex(),
            From:      msg.From().Hex(),
            To:        toAddress,
            Value:     tx.Value(),
            Gas:       tx.Gas(),
            GasPrice:  tx.GasPrice(),
            Nonce:     tx.Nonce(),
            Data:      hexutil.Encode(tx.Data()),
            BlockNumber: blockNumber,
            BlockHash: block.Hash().Hex(),
        }

        transactions = append(transactions, detail)
    }

    return transactions, nil
}

```

```
}
```

7. 数据结构化存储

7.1 MySQL存储方案

功能说明：将区块链数据持久化到MySQL数据库。

设计原则：

1. 数据规范化：避免数据冗余，确保数据一致性
2. 索引优化：为常用查询字段建立合适的索引
3. 分区策略：大表按时间或区块高度分区提升性能
4. 约束设计：使用唯一约束防止重复数据插入

表结构说明：

- **account_balances**：存储账户原生代币余额，支持历史查询
- **erc20_balances**：存储ERC20代币余额，包含代币元信息
- **transactions**：交易记录表，包含完整的交易信息和状态
- **blocks**：区块信息表，存储区块头部数据
- **contract_events**：合约事件表，支持复杂的事件查询

// 数据库表结构定义

-- 账户余额表

```
CREATE TABLE `account_balances` (  
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,  
  `address` varchar(42) NOT NULL COMMENT '账户地址',  
  `balance` varchar(78) NOT NULL COMMENT '余额 (wei)',  
  `balance_ether` decimal(65,18) DEFAULT NULL COMMENT '余额 (ether)',  
  `block_number` bigint(20) unsigned NOT NULL COMMENT '查询区块高度',  
  `updated_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `idx_address` (`address`),  
  KEY `idx_block_number` (`block_number`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='账户余额表';
```

-- ERC20代币余额表

```
CREATE TABLE `erc20_balances` (  
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,  
  `token_address` varchar(42) NOT NULL COMMENT '代币合约地址',  
  `holder_address` varchar(42) NOT NULL COMMENT '持有者地址',  
  `balance` varchar(78) NOT NULL COMMENT '余额',  
  `decimals` tinyint(3) unsigned NOT NULL COMMENT '精度',  
  `symbol` varchar(20) DEFAULT NULL COMMENT '代币符号',  
  `name` varchar(100) DEFAULT NULL COMMENT '代币名称',  
  `block_number` bigint(20) unsigned NOT NULL COMMENT '查询区块高度',  
  `updated_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),
```

```

    UNIQUE KEY `idx_token_holder` (`token_address`,`holder_address`),
    KEY `idx_holder` (`holder_address`),
    KEY `idx_block_number` (`block_number`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='ERC20代币余额表';

```

-- 交易记录表

```

CREATE TABLE `transactions` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `tx_hash` varchar(66) NOT NULL COMMENT '交易哈希',
  `from_address` varchar(42) NOT NULL COMMENT '发送者地址',
  `to_address` varchar(42) DEFAULT NULL COMMENT '接收者地址',
  `value` varchar(78) NOT NULL COMMENT '转账金额 (wei) ',
  `gas_limit` bigint(20) unsigned NOT NULL COMMENT 'Gas限制',
  `gas_price` varchar(78) NOT NULL COMMENT 'Gas价格',
  `gas_used` bigint(20) unsigned DEFAULT NULL COMMENT '实际Gas消耗',
  `nonce` bigint(20) unsigned NOT NULL COMMENT 'Nonce',
  `data` text COMMENT '交易数据',
  `block_number` bigint(20) unsigned DEFAULT NULL COMMENT '区块号',
  `block_hash` varchar(66) DEFAULT NULL COMMENT '区块哈希',
  `status` tinyint(3) unsigned DEFAULT NULL COMMENT '状态 (1:成功, 0:失败) ',
  `confirmations` int(11) unsigned DEFAULT 0 COMMENT '确认数',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `idx_tx_hash` (`tx_hash`),
  KEY `idx_from` (`from_address`),
  KEY `idx_to` (`to_address`),
  KEY `idx_block` (`block_number`),
  KEY `idx_status` (`status`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='交易记录表';

```

-- 区块信息表

```

CREATE TABLE `blocks` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `block_number` bigint(20) unsigned NOT NULL COMMENT '区块号',
  `block_hash` varchar(66) NOT NULL COMMENT '区块哈希',
  `parent_hash` varchar(66) NOT NULL COMMENT '父区块哈希',
  `timestamp` bigint(20) unsigned NOT NULL COMMENT '时间戳',
  `miner` varchar(42) NOT NULL COMMENT '矿工地址',
  `difficulty` varchar(78) DEFAULT NULL COMMENT '难度',
  `gas_limit` bigint(20) unsigned NOT NULL COMMENT 'Gas限制',
  `gas_used` bigint(20) unsigned NOT NULL COMMENT 'Gas使用量',
  `tx_count` int(11) unsigned NOT NULL COMMENT '交易数量',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `idx_block_number` (`block_number`),
  UNIQUE KEY `idx_block_hash` (`block_hash`),
  KEY `idx_timestamp` (`timestamp`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='区块信息表';

```

-- 合约事件表

```

CREATE TABLE `contract_events` (

```

```

`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`contract_address` varchar(42) NOT NULL COMMENT '合约地址',
`event_name` varchar(100) NOT NULL COMMENT '事件名称',
`event_signature` varchar(66) NOT NULL COMMENT '事件签名',
`block_number` bigint(20) unsigned NOT NULL COMMENT '区块号',
`tx_hash` varchar(66) NOT NULL COMMENT '交易哈希',
`log_index` int(11) unsigned NOT NULL COMMENT '日志索引',
`event_data` json DEFAULT NULL COMMENT '事件数据 (JSON)',
`removed` tinyint(1) NOT NULL DEFAULT 0 COMMENT '是否被移除 (分叉)',
`created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (`id`),
KEY `idx_contract` (`contract_address`),
KEY `idx_event_name` (`event_name`),
KEY `idx_block` (`block_number`),
KEY `idx_tx_hash` (`tx_hash`),
KEY `idx_removed` (`removed`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='合约事件表';

```

7.2 GORM数据模型

功能说明：使用GORM ORM框架的数据模型定义。

模型设计要点：

1. 标签定义：使用GORM标签定义字段属性、索引和约束
2. 类型映射：Go类型与MySQL类型的合理映射
3. 关联关系：定义表之间的外键关联关系
4. 钩子方法：实现BeforeCreate、AfterUpdate等生命周期钩子

字段说明：

- 使用 `uint64` 存储区块号和时间戳避免溢出
- 大数值使用 `string` 类型存储保持精度
- JSON字段存储复杂的事件数据
- 时间字段使用 `time.Time` 类型便于处理

```

import (
    "time"
    "gorm.io/gorm"
)

// AccountBalance 账户余额模型
type AccountBalance struct {
    ID            uint64      `gorm:"primaryKey;autoIncrement" json:"id"`
    Address       string      `gorm:"type:varchar(42);uniqueIndex;not null" json:"address"`
    Balance       string      `gorm:"type:varchar(78);not null" json:"balance"`
    BalanceEther  string      `gorm:"type:decimal(65,18)" json:"balance_ether"`
    BlockNumber   uint64      `gorm:"not null;index" json:"block_number"`
    UpdatedAt     time.Time   `json:"updated_at"`
}

func (AccountBalance) TableName() string {

```

```

        return "account_balances"
    }

// ERC20Balance ERC20余额模型
type ERC20Balance struct {
    ID          uint64      `gorm:"primaryKey;autoIncrement" json:"id"`
    TokenAddress string      `gorm:"type:varchar(42);not
null;uniqueIndex:idx_token_holder,priority:1" json:"token_address"`
    HolderAddress string    `gorm:"type:varchar(42);not
null;uniqueIndex:idx_token_holder,priority:2;index:idx_holder" json:"holder_address"`
    Balance     string      `gorm:"type:varchar(78);not null" json:"balance"`
    Decimals    uint8       `gorm:"not null" json:"decimals"`
    Symbol      string      `gorm:"type:varchar(20)" json:"symbol"`
    Name        string      `gorm:"type:varchar(100)" json:"name"`
    BlockNumber uint64      `gorm:"not null;index" json:"block_number"`
    UpdatedAt   time.Time   `json:"updated_at"`
}

func (ERC20Balance) TableName() string {
    return "erc20_balances"
}

// Transaction 交易模型
type Transaction struct {
    ID          uint64      `gorm:"primaryKey;autoIncrement" json:"id"`
    TxHash      string      `gorm:"type:varchar(66);uniqueIndex;not null" json:"tx_hash"`
    FromAddress string      `gorm:"type:varchar(42);not null;index:idx_from"
json:"from_address"`
    ToAddress   string      `gorm:"type:varchar(42);index:idx_to" json:"to_address"`
    Value       string      `gorm:"type:varchar(78);not null" json:"value"`
    GasLimit    uint64      `gorm:"not null" json:"gas_limit"`
    GasPrice    string      `gorm:"type:varchar(78);not null" json:"gas_price"`
    GasUsed     uint64      `json:"gas_used"`
    Nonce       uint64      `gorm:"not null" json:"nonce"`
    Data        string      `gorm:"type:text" json:"data"`
    BlockNumber uint64      `gorm:"index:idx_block" json:"block_number"`
    BlockHash   string      `gorm:"type:varchar(66)" json:"block_hash"`
    Status      uint8       `gorm:"index:idx_status" json:"status"`
    Confirmations uint64      `gorm:"default:0" json:"confirmations"`
    CreatedAt   time.Time   `json:"created_at"`
    UpdatedAt   time.Time   `json:"updated_at"`
}

func (Transaction) TableName() string {
    return "transactions"
}

// Block 区块模型
type Block struct {
    ID          uint64      `gorm:"primaryKey;autoIncrement" json:"id"`
    BlockNumber uint64      `gorm:"uniqueIndex;not null" json:"block_number"`
    BlockHash   string      `gorm:"type:varchar(66);uniqueIndex;not null" json:"block_hash"`

```

```

    ParentHash    string    `gorm:"type:varchar(66);not null" json:"parent_hash"`
    Timestamp     uint64    `gorm:"not null;index" json:"timestamp"`
    Miner         string    `gorm:"type:varchar(42);not null" json:"miner"`
    Difficulty     string    `gorm:"type:varchar(78)" json:"difficulty"`
    GasLimit       uint64    `gorm:"not null" json:"gas_limit"`
    GasUsed        uint64    `gorm:"not null" json:"gas_used"`
    TxCount        uint      `gorm:"not null" json:"tx_count"`
    CreatedAt      time.Time `json:"created_at"`
}

func (Block) TableName() string {
    return "blocks"
}

// ContractEvent 合约事件模型
type ContractEvent struct {
    ID                uint64          `gorm:"primaryKey;autoIncrement" json:"id"`
    ContractAddress   string          `gorm:"type:varchar(42);not null;index:idx_contract" json:"contract_address"`
    EventName         string          `gorm:"type:varchar(100);not null;index:idx_event_name" json:"event_name"`
    EventSignature    string          `gorm:"type:varchar(66);not null" json:"event_signature"`
    BlockNumber       uint64          `gorm:"not null;index:idx_block" json:"block_number"`
    TxHash            string          `gorm:"type:varchar(66);not null;index:idx_tx_hash" json:"tx_hash"`
    LogIndex          uint           `gorm:"not null" json:"log_index"`
    EventData         json.RawMessage `gorm:"type:json" json:"event_data"`
    Removed           bool           `gorm:"default:false;index:idx_removed" json:"removed"`
    CreatedAt         time.Time      `json:"created_at"`
}

func (ContractEvent) TableName() string {
    return "contract_events"
}

```

7.3 数据存储服务

功能说明：封装数据库操作的服务层。

服务层设计：

1. **操作封装**：将复杂的数据库操作封装为简单的方法调用
2. **事务管理**：支持事务操作确保数据一致性
3. **批量处理**：提供批量插入和更新方法提升性能
4. **错误处理**：统一的错误处理和日志记录

核心功能：

- **CRUD操作**：提供完整的增删改查功能
- **条件查询**：支持复杂的条件查询和排序

- 分页查询：大数据量的分页处理
- 统计分析：提供数据统计和分析方法

```
// DataStore 数据存储服务
type DataStore struct {
    db *gorm.DB
}

// NewDataStore 创建数据存储服务
func NewDataStore(db *gorm.DB) *DataStore {
    return &DataStore{db: db}
}

// SaveTransaction 保存交易记录
func (ds *DataStore) SaveTransaction(tx *Transaction) error {
    return ds.db.Create(tx).Error
}

// UpdateTransactionStatus 更新交易状态
func (ds *DataStore) UpdateTransactionStatus(txHash string, status uint8, gasUsed,
confirmations uint64) error {
    return ds.db.Model(&Transaction{}).
        Where("tx_hash = ?", txHash).
        Updates(map[string]interface{}{
            "status":      status,
            "gas_used":    gasUsed,
            "confirmations": confirmations,
        }).Error
}

// GetTransactionByHash 根据哈希查询交易
func (ds *DataStore) GetTransactionByHash(txHash string) (*Transaction, error) {
    var tx Transaction
    err := ds.db.Where("tx_hash = ?", txHash).First(&tx).Error
    if err != nil {
        return nil, err
    }
    return &tx, nil
}

// SaveBlock 保存区块信息
func (ds *DataStore) SaveBlock(block *Block) error {
    return ds.db.Create(block).Error
}

// GetBlockByNumber 根据区块号查询区块
func (ds *DataStore) GetBlockByNumber(blockNumber uint64) (*Block, error) {
    var block Block
    err := ds.db.Where("block_number = ?", blockNumber).First(&block).Error
    if err != nil {
        return nil, err
    }
}
```

```

    return &block, nil
}

// SaveContractEvent 保存合约事件
func (ds *DataStore) SaveContractEvent(event *ContractEvent) error {
    return ds.db.Create(event).Error
}

// QueryContractEvents 查询合约事件
func (ds *DataStore) QueryContractEvents(contractAddr string, eventName string, fromBlock,
toBlock uint64) ([]*ContractEvent, error) {
    var events []*ContractEvent
    query := ds.db.Where("contract_address = ?", contractAddr).
        Where("block_number >= ? AND block_number <= ?", fromBlock, toBlock)

    if eventName != "" {
        query = query.Where("event_name = ?", eventName)
    }

    err := query.Order("block_number ASC, log_index ASC").Find(&events).Error
    return events, err
}

// UpdateAccountBalance 更新账户余额
func (ds *DataStore) UpdateAccountBalance(balance *AccountBalance) error {
    return ds.db.Save(balance).Error
}

// BatchSaveTransactions 批量保存交易
func (ds *DataStore) BatchSaveTransactions(txs []*Transaction) error {
    return ds.db.CreateInBatches(txs, 100).Error
}

```

8. 生产环境最佳实践

8.1 错误处理与重试机制

功能说明： 实现智能的错误处理和指数退避重试机制。

重试策略：

1. **错误分类：** 区分可重试错误（网络超时）和不可重试错误（参数错误）
2. **指数退避：** 重试间隔按指数增长，避免对服务器造成压力
3. **最大限制：** 设置最大重试次数和最大退避时间防止无限重试
4. **快速失败：** 对于明确的业务错误立即返回不进行重试

适用场景：

- 网络连接不稳定时的API调用
- 区块链节点临时不可用
- 数据库连接池满时的操作重试

- 第三方服务的限流处理

```
// RetryConfig 重试配置
type RetryConfig struct {
    MaxRetries      int           // 最大重试次数
    InitialBackoff  time.Duration // 初始退避时间
    MaxBackoff      time.Duration // 最大退避时间
    Multiplier      float64       // 退避倍数
}

// RetryWithBackoff 带退避的重试机制
func RetryWithBackoff(config *RetryConfig, fn func() error) error {
    backoff := config.InitialBackoff

    for attempt := 0; attempt <= config.MaxRetries; attempt++ {
        err := fn()
        if err == nil {
            return nil
        }

        if attempt == config.MaxRetries {
            return fmt.Errorf("达到最大重试次数 %d: %w", config.MaxRetries, err)
        }

        // 判断是否应该重试
        if !shouldRetry(err) {
            return err
        }

        log.Printf("操作失败, %v 后重试 (attempt %d/%d): %v",
            backoff, attempt+1, config.MaxRetries, err)

        time.Sleep(backoff)

        // 指数退避
        backoff = time.Duration(float64(backoff) * config.Multiplier)
        if backoff > config.MaxBackoff {
            backoff = config.MaxBackoff
        }
    }

    return fmt.Errorf("重试失败")
}

// shouldRetry 判断错误是否应该重试
func shouldRetry(err error) bool {
    if err == nil {
        return false
    }
}

// 网络错误应该重试
if strings.Contains(err.Error(), "connection refused") ||
```

```

    strings.Contains(err.Error(), "timeout") ||
    strings.Contains(err.Error(), "EOF") {
        return true
    }

    // 节点临时不可用
    if strings.Contains(err.Error(), "503") ||
        strings.Contains(err.Error(), "429") {
        return true
    }

    return false
}

```

8.2 连接池管理

```

// ClientPool RPC客户端连接池
type ClientPool struct {
    clients []*ethclient.Client
    config  *ChainConfig
    current atomic.Int32
    mu      sync.RWMutex
}

// NewClientPool 创建客户端连接池
func NewClientPool(config *ChainConfig) (*ClientPool, error) {
    pool := &ClientPool{
        clients: make([]*ethclient.Client, 0, len(config.RPCUrls)),
        config:  config,
    }

    // 连接所有节点
    for _, rpcURL := range config.RPCUrls {
        ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
        client, err := ethclient.DialContext(ctx, rpcURL)
        cancel()

        if err != nil {
            log.Printf("连接节点失败 %s: %v", rpcURL, err)
            continue
        }

        pool.clients = append(pool.clients, client)
        log.Printf("成功连接节点: %s", rpcURL)
    }

    if len(pool.clients) == 0 {
        return nil, fmt.Errorf("没有可用的RPC节点")
    }

    return pool, nil
}

```

```

}

// Get 获取一个客户端 (轮询)
func (p *ClientPool) Get() *ethclient.Client {
    p.mu.RLock()
    defer p.mu.RUnlock()

    if len(p.clients) == 0 {
        return nil
    }

    idx := p.current.Add(1) % int32(len(p.clients))
    return p.clients[idx]
}

// Close 关闭所有连接
func (p *ClientPool) Close() {
    p.mu.Lock()
    defer p.mu.Unlock()

    for _, client := range p.clients {
        client.Close()
    }
    p.clients = nil
}

```

8.3 分叉检测与处理

```

// ForkDetector 分叉检测器
type ForkDetector struct {
    client      *ethclient.Client
    blockHashes map[uint64]string // 区块号 -> 区块哈希
    mu          sync.RWMutex
}

// NewForkDetector 创建分叉检测器
func NewForkDetector(client *ethclient.Client) *ForkDetector {
    return &ForkDetector{
        client:      client,
        blockHashes: make(map[uint64]string),
    }
}

// CheckFork 检测分叉
// 返回: 是否发生分叉, 分叉起始区块号
func (fd *ForkDetector) CheckFork(blockNumber uint64, blockHash string) (bool, uint64, error) {
    fd.mu.Lock()
    defer fd.mu.Unlock()

    // 检查是否有该区块的记录

```

```

    if existingHash, exists := fd.blockHashes[blockNumber]; exists {
        if existingHash != blockHash {
            // 发现分叉
            log.Printf("检测到分叉: 区块 %d, 旧哈希 %s, 新哈希 %s",
                blockNumber, existingHash, blockHash)

            // 找到分叉起始点
            forkStart := fd.findForkStart(blockNumber)

            // 清理分叉后的数据
            fd.cleanupAfterFork(forkStart)

            return true, forkStart, nil
        }
    }

    // 记录区块哈希
    fd.blockHashes[blockNumber] = blockHash

    return false, 0, nil
}

// findForkStart 找到分叉起始区块
func (fd *ForkDetector) findForkStart(currentBlock uint64) uint64 {
    ctx := context.Background()

    for blockNum := currentBlock; blockNum > 0; blockNum-- {
        block, err := fd.client.BlockByNumber(ctx, big.NewInt(int64(blockNum)))
        if err != nil {
            log.Printf("获取区块失败 %d: %v", blockNum, err)
            continue
        }

        savedHash, exists := fd.blockHashes[blockNum]
        if exists && savedHash == block.Hash().Hex() {
            // 找到分叉前的最后一个正确区块
            return blockNum + 1
        }
    }

    return 1
}

// cleanupAfterFork 清理分叉后的数据
func (fd *ForkDetector) cleanupAfterFork(forkStart uint64) {
    for blockNum := range fd.blockHashes {
        if blockNum >= forkStart {
            delete(fd.blockHashes, blockNum)
        }
    }
}

```

8.4 监控与日志

```
// Metrics 监控指标
type Metrics struct {
    RequestCount    int64
    ErrorCount      int64
    AvgResponseTime time.Duration
    LastUpdateTime  time.Time
    mu              sync.RWMutex
}

// Record 记录请求
func (m *Metrics) Record(duration time.Duration, err error) {
    m.mu.Lock()
    defer m.mu.Unlock()

    m.RequestCount++
    if err != nil {
        m.ErrorCount++
    }

    // 计算平均响应时间
    m.AvgResponseTime = (m.AvgResponseTime*time.Duration(m.RequestCount-1) + duration) /
time.Duration(m.RequestCount)
    m.LastUpdateTime = time.Now()
}

// GetStats 获取统计信息
func (m *Metrics) GetStats() map[string]interface{} {
    m.mu.RLock()
    defer m.mu.RUnlock()

    errorRate := float64(0)
    if m.RequestCount > 0 {
        errorRate = float64(m.ErrorCount) / float64(m.RequestCount) * 100
    }

    return map[string]interface{}{
        "request_count":    m.RequestCount,
        "error_count":      m.ErrorCount,
        "error_rate":       fmt.Sprintf("%.2f%%", errorRate),
        "avg_response_time": m.AvgResponseTime.String(),
        "last_update":      m.LastUpdateTime.Format(time.RFC3339),
    }
}

// Logger 日志工具
type Logger struct {
    logger *log.Logger
    level  string
}
```

```

// NewLogger 创建日志工具
func NewLogger(level string) *Logger {
    return &Logger{
        logger: log.New(os.Stdout, "", log.LstdFlags|log.Lshortfile),
        level: level,
    }
}

// Info 信息日志
func (l *Logger) Info(format string, v ...interface{}) {
    l.logger.Printf("[INFO] "+format, v...)
}

// Error 错误日志
func (l *Logger) Error(format string, v ...interface{}) {
    l.logger.Printf("[ERROR] "+format, v...)
}

// Debug 调试日志
func (l *Logger) Debug(format string, v ...interface{}) {
    if l.level == "debug" {
        l.logger.Printf("[DEBUG] "+format, v...)
    }
}

```

9. 完整示例：NFT交易监控系统

```

package main

import (
    "context"
    "log"
    "math/big"
    "time"

    "github.com/ethereum/go-ethereum/ethclient"
    "gorm.io/driver/mysql"
    "gorm.io/gorm"
)

// NFTMonitor NFT交易系统
type NFTMonitor struct {
    client      *ethclient.Client
    dataStore   *DataStore
    detector    *ForkDetector
    metrics     *Metrics
    logger      *Logger
}

```

```

// NewNFTMonitor 创建NFT监控系统
func NewNFTMonitor(rpcURL, dbDSN string) (*NFTMonitor, error) {
    // 连接以太坊节点
    client, err := ethclient.Dial(rpcURL)
    if err != nil {
        return nil, err
    }

    // 连接数据库
    db, err := gorm.Open(mysql.Open(dbDSN), &gorm.Config{})
    if err != nil {
        return nil, err
    }

    // 自动迁移表结构
    db.AutoMigrate(&Transaction{}, &Block{}, &ContractEvent{})

    return &NFTMonitor{
        client:    client,
        dataStore: NewDataStore(db),
        detector:  NewForkDetector(client),
        metrics:   &Metrics{},
        logger:    NewLogger("info"),
    }, nil
}

// Start 启动监控
func (nm *NFTMonitor) Start() {
    nm.logger.Info("NFT监控系统启动")

    ticker := time.NewTicker(3 * time.Second)
    defer ticker.Stop()

    for range ticker.C {
        nm.processLatestBlock()
    }
}

// processLatestBlock 处理最新区块
func (nm *NFTMonitor) processLatestBlock() {
    start := time.Now()
    var err error

    defer func() {
        nm.metrics.Record(time.Since(start), err)
    }()

    // 获取最新区块
    blockInfo, err := GetLatestBlock(nm.client)
    if err != nil {
        nm.logger.Error("获取最新区块失败: %v", err)
        return
    }
}

```

```

}

// 检测分叉
isFork, forkStart, err := nm.detector.CheckFork(blockInfo.Number, blockInfo.Hash)
if err != nil {
    nm.logger.Error("检测分叉失败: %v", err)
    return
}

if isFork {
    nm.logger.Info("处理分叉, 起始区块: %d", forkStart)
    nm.handleFork(forkStart)
}

// 保存区块
block := &Block{
    BlockNumber: blockInfo.Number,
    BlockHash:   blockInfo.Hash,
    ParentHash:  blockInfo.ParentHash,
    Timestamp:   blockInfo.Timestamp,
    Miner:       blockInfo.Miner,
    GasLimit:    blockInfo.GasLimit,
    GasUsed:     blockInfo.GasUsed,
    TxCount:     uint(blockInfo.TxCount),
}

if err := nm.dataStore.SaveBlock(block); err != nil {
    nm.logger.Error("保存区块失败: %v", err)
    return
}

nm.logger.Info("处理区块 # %d, 交易数: %d", blockInfo.Number, blockInfo.TxCount)
}

// handleFork 处理分叉
func (nm *NFTMonitor) handleFork(forkStart uint64) {
    nm.logger.Info("回滚分叉数据, 起始区块: %d", forkStart)
    // 实现分叉数据回滚逻辑
}

func main() {
    rpcURL := "https://eth-mainnet.g.alchemy.com/v2/YOUR_API_KEY"
    dbDSN := "user:password@tcp(127.0.0.1:3306)/blockchain?charset=utf8mb4&parseTime=True&loc=Local"

    monitor, err := NewNFTMonitor(rpcURL, dbDSN)
    if err != nil {
        log.Fatal(err)
    }

    monitor.Start()
}

```

